Université Abdelmalek Essaâdi

École Nationale des Sciences Appliquées – Al Hoceima Filière : ID1 – S2

Année universitaire : 2025 - 2026

Compte Rendu

TP N°4 :

Les principes de la POO :

Héritage, Polymorphisme, Classes abstraites et Interfaces

Objectif :

L'objectif de ce TP est de comprendre les principes de bases de la programmation orienté objet.

Réalisé par : Hajar LEBYED

Pr : Mohamed CHERRADI

Exercice 1 (*Initialisation et spécialisation d'une entité*):

Dans cet exercice, j'ai modélisé des individus et des étudiants dans le cadre d'un système de gestion académique, en appliquant le principe d'héritage.

Classe Personne :

J'ai créé une classe Personne dans le package Ex1. J'ai déclaré trois attributs protégés : nom, prenom et age de types String, String et int. J'ai utilisé le modificateur protected pour permettre l'accès à ces attributs depuis les classes filles.

J'ai ensuite créé un constructeur paramétré qui prend en paramètres nom, prenom et age, et qui les initialise via this.

J'ai implémenté les méthodes suivantes :

- **afficherInformations() :** qui affiche le nom, le prénom et l'âge de la personne.
- **estMajeur() :** qui retourne true si l'âge est supérieur ou égal à 18.
- **sePresenter() :** qui retourne une chaîne de caractères décrivant la personne.

```java
package Ex1;

public class Personne {
    protected String nom;
    protected String prenom;
    protected int age;

    Personne(String nom,String prenom,int age){
        this.nom=nom;
        this.prenom=prenom;
        this.age=age;
    }

    public void afficherInformations() {
        System.out.println("le nom : "+nom);
        System.out.println("le prenom : "+prenom);
        System.out.println("l'age : "+age);
    }

    public boolean estMajeur() {
        return age>=18;
    }

    public String sePresenter() {
        return "je suis "+prenom+" "+nom+" j'ai "+age+"ans".
    }
}
```

Classe Etudiant :

J'ai créé une classe Etudiant qui hérite de la classe Personne en utilisant le mot-clé **extends**. J'ai ajouté deux attributs supplémentaires : **niveauEtude** de type String et moyenne de type double.

J'ai créé un constructeur paramétré qui prend cinq paramètres. J'ai utilisé super(nom, prenom, age) pour appeler le constructeur de la classe parente Personne et initialiser les attributs hérités, puis j'ai initialisé **niveauEtude** et moyenne via this.

J'ai implémenté les méthodes spécifiques suivantes :

- calculerMention() : qui affiche la mention selon la moyenne — "Très bien" si la moyenne est supérieure ou égale à 16, "Bien" si elle est supérieure ou égale à 14, "Assez bien" si elle est supérieure ou égale à 12, et "Ajourné" sinon.
- estAdmis() : qui retourne true si la moyenne est supérieure ou égale à 12.
- afficherProfil() : qui appelle afficherInformations() héritée de Personne, puis affiche le niveau d'étude et la moyenne.

```java
package Ex1;

public class Etudiant extends Personne {
    private String niveauEtude;
    private double moyenne;

    Etudiant (String nom,String prenom,int age,String niveauEtude,double moyenne){
        super(nom,prenom,age);
        this.niveauEtude=niveauEtude;
        this.moyenne=moyenne;
    }

    public void calculerMention() {
        if(moyenne>=16)
            System.out.println("Mention : tres bien");
        else if(moyenne>=14)
            System.out.println("Mention : bien");
        else if(moyenne>=12)
            System.out.println("Mention : Assez bien");
        else if(moyenne<12)
            System.out.println("Mention : ajourné");
    }

    public boolean estAdmis(){
        return moyenne>=12;
    }

    public void afficherProfil() {
        afficherInformations();
        System.out.println("niveau d'étude : "+niveauEtude);
        System.out.println("moyenne : "+moyenne);
    }
}
```

Classe Main :

Dans la classe Main, j'ai créé un objet e1 de type Etudiant avec les valeurs : nom "hajar", prénom "lebyed", âge 21, niveau "Cycle" et moyenne 15.6.

J'ai ensuite appelé les méthodes suivantes sur cet objet :

- **sePresenter()** : héritée de Personne, pour afficher une présentation de l'étudiant.
- **afficherProfil()** : pour afficher les informations académiques.
- **estMajeur()** : pour vérifier si l'étudiant est majeur.
- **estAdmis()** : pour vérifier si l'étudiant est admis.
- **calculerMention()** : pour afficher la mention correspondant à sa moyenne.

```java
package Ex1;

public class Main {
    public static void main(String[] args) {
        Etudiant e1 = new Etudiant("hajar","lebyed",21,"Cycle",15.6);
        e1.sePresenter();
        e1.afficherProfil();
        if(e1.estMajeur())
            System.out.println("Est majeur ?  ,oui");
        else
            System.out.println("Est majeur ?  ,non");

        if(e1.estAdmis())
            System.out.println("Est admis ?  ,oui");
        else
            System.out.println("Est admis ?  ,non");
        e1.calculerMention();
    }
}
```

Résultat :

```
le nom : hajar
le prenom : lebyed
l'age : 21
niveau d'étude : Cycle
moyenne : 15.6
Est majeur ?  ,oui
Est admis ?  ,oui
Mention : bien
```

Exercice 2 : Constructeur implicite vs explicite

Dans cet exercice, j'ai modélisé un système de gestion de véhicules en mettant en évidence l'importance des constructeurs dans une hiérarchie d'héritage.

Classe Vehicule

J'ai créé une classe Vehicule avec un attribut protégé marque. J'ai défini deux constructeurs : un constructeur par défaut qui initialise marque à null, et un constructeur paramétré. J'ai aussi implémenté les méthodes **afficherMarque()**, **demarrer()** et **arreter().**

```java
package Ex2;

public class Vehicule {
    protected String marque;

    public Vehicule() {
        this.marque = null;
    }

    public Vehicule(String marque){
        this.marque=marque;
    }

    public String afficherMarque() {
        return "la marque : "+marque;
    }

    public void demarrer() {
        System.out.println("la voiture demarre");
    }

    public void arreter() {
        System.out.println("la voiture s'arrete");
    }

}
```

Classe Voiture : sans appel explicite au constructeur parent

J'ai créé une classe Voiture héritant de Vehicule avec les attributs nombre_portes et carburant. Dans cette version, je n'ai pas appelé super(marque), donc Java a invoqué automatiquement le constructeur par défaut, ce qui a laissé marque à null.

```java
package Ex2;

public class Main {
    public static void main(String[] args) {
        Voiture v = new Voiture(4,"Gasoil");
        System.out.println(v.afficherMarque());
        v.demarrer();
        v.arreter();
        v.afficherDetails();
        v.klaxonner();
    }
}
```

Classe Voiture : avec appel explicite

J'ai créé une deuxième version où j'ai utilisé super(marque) dans le constructeur, ce qui a permis d'initialiser correctement l'attribut marque hérité de Vehicule.

```java
package Ex2;

public class Voiture extends Vehicule {
    private int nombre_portes;
    private String carburant;

    public Voiture (int nombre_portes,String carburant) {
        //implicitement
        this.nombre_portes=nombre_portes;
        this.carburant=carburant;
    }

    public void afficherDetails() {
        System.out.println(afficherMarque());
        System.out.println("Portes :"+nombre_portes+" | carburant : "+carburant);
    }

    public void klaxonner() {
        System.out.println(marque+" klaxonne");
    }
}
```

Résultat :

```
<terminated> Main (1) [Java Application] C:\Use
la marque : null
la voiture demarre
la voiture s'arrete
la marque : null
Portes :4 | carburant : Gasoil
null klaxonne
```

Exercice 3 : Héritage – Système bancaire

Dans cet exercice, j'ai conçu un système de gestion de comptes bancaires en appliquant le principe d'héritage.

Classe Compte :

J'ai créé une classe Compte avec un attribut protégé solde. J'ai implémenté les méthodes deposer() pour ajouter un montant, retirer() pour retirer un montant avec vérification du solde, consulterSolde() pour afficher le solde, et afficher() pour afficher les informations du compte.

```java
package Ex3;

public class Compte {
    protected double solde;

    public Compte(double solde) {
        this.solde=solde;
    }

    public void deposer(double montant) {
        solde+=montant;
        System.out.println("Dépôt de "+montant+" effectué.");
    }

    public void retirer(double montant) {
        if (montant<=solde) {
            solde-=montant;
            System.out.println("Retrait de "+montant+" effectué.");
        } else {
            System.out.println("Solde insuffisant.");
        }
    }

    public void consulterSolde() {
        System.out.println("Solde actuel "+solde);
    }

    public void afficher() {
        System.out.println("Compte - Solde : "+solde);
    }
}
```

Classe CompteCourant

J'ai créé une classe CompteCourant qui hérite de Compte avec un attribut supplémentaire decouvertMax. J'ai redéfini la méthode retirer() pour autoriser les retraits jusqu'au découvert maximum, et j'ai ajouté autoriserDecouvert() pour afficher le découvert autorisé, et calculerFrais() pour calculer les frais bancaires en cas de solde négatif. J'ai également redéfini afficher() pour afficher les informations spécifiques au compte courant.

```java
package Ex3;

public class CompteCourant extends Compte {
    private double decouvertMax;

    public CompteCourant(double solde, double decouvertMax) {
        super(solde);
        this.decouvertMax = decouvertMax;
    }

    public void autoriserDecouvert() {
        System.out.println("Découvert autorisé jusqu'à : " + decouvertMax);
    }

    public void calculerFrais() {
        double frais=solde<0 ? Math.abs(solde)*0.05 : 0;
        System.out.println("Frais bancaires : "+frais);
    }

    public void retirer(double montant) {
        if (montant <= solde + decouvertMax) {
            solde-=montant;
            System.out.println("Retrait de " +montant+ " effectué.");
        } else {
            System.out.println("Dépassement du découvert autorisé.");
        }
    }

    public void afficher() {
        System.out.println("Compte Courant - Solde : "+solde+" | Découvert max : " +decouvertMax);
    }
}
```

Classe CompteEpargne

J'ai créé une classe CompteEpargne qui hérite de Compte avec un attribut tauxInteret. J'ai ajouté calculerInterets() qui retourne les intérêts calculés selon le taux, et ajouterInterets() qui ajoute les intérêts au solde. J'ai aussi redéfini afficher() pour afficher le solde et le taux d'intérêt.

```java
package Ex3;

public class CompteEpargne extends Compte {
    private double tauxInteret;

    public CompteEpargne(double solde, double tauxInteret) {
        super(solde);
        this.tauxInteret = tauxInteret;
    }

    public double calculerInterets() {
        return solde * tauxInteret / 100;
    }

    public void ajouterInterets() {
        double interets = calculerInterets();
        solde += interets;
        System.out.println("Intérêts ajoutés : " + interets);
    }

    public void afficher() {
        System.out.println("Compte Épargne - Solde : " + solde + " | Taux : " + tauxInteret + "%");
    }
}
```

Classe Main

Dans le Main, j'ai créé un objet CompteCourant avec un solde de 1000 et un découvert de 500, et un objet CompteEpargne avec un solde de 5000 et un taux de 3.5%. J'ai testé toutes les méthodes pour vérifier le bon fonctionnement de l'héritage et de la redéfinition des méthodes.

```java
package Ex3;

public class Main {
    public static void main(String[] args) {
        CompteCourant cc = new CompteCourant(1000, 500);
        cc.afficher();
        cc.deposer(200);
        cc.retirer(1600);
        cc.autoriserDecouvert();
        cc.calculerFrais();
        cc.consulterSolde();

        CompteEpargne ce = new CompteEpargne(5000, 3.5);
        ce.afficher();
        ce.deposer(1000);
        ce.ajouterInterets();
        ce.consulterSolde();
    }
}
```

Résultat :

```
Compte Courant - Solde : 1000.0 | Découvert max : 500.0
Dépôt de 200.0 effectué.
Retrait de 1600.0 effectué.
Découvert autorisé jusqu'à : 500.0
Frais bancaires : 20.0
Solde actuel -400.0
Compte Épargne - Solde : 5000.0 | Taux : 3.5%
Dépôt de 1000.0 effectué.
Intérêts ajoutés : 210.0
Solde actuel 6210.0
```

Exercice 4 : Polymorphisme – Animaux

Dans cet exercice, j'ai modélisé des animaux avec des comportements dynamiques en appliquant le principe du polymorphisme.

Classe Animal

J'ai créé une classe Animal avec trois méthodes : crier() qui affiche un cri générique, manger() et dormir() qui affichent les comportements de base de l'animal.

```java
package Ex4;

public class Animal {

    public void crier() {
            System.out.println("L'animal fait un cri");
    }
    public void manger() {
        System.out.println("L'animal est en train de manger.");
    }
    public void dormir() {
        System.out.println("L'animal est en train de dormir.");
    }
}
```

Classe Chien

J'ai créé une classe Chien qui hérite de Animal. J'ai redéfini la méthode crier() pour afficher un cri spécifique au chien. J'ai également ajouté deux méthodes spécifiques : garder() et jouer().

```java
package Ex4;
TP4/src/Ex4/Animal.java
public class Chien extends Animal{

    public void crier() {
        System.out.println("Le chien fait un cri.");
    }
    public void garder() {
        System.out.println("Le chien garde.");
    }
    public void jouer() {
        System.out.println("Le chien joue");
    }

}
```

Classe Chat

J'ai créé une classe Chat qui hérite de Animal. J'ai redéfini la méthode crier() pour afficher un cri spécifique au chat. J'ai aussi ajouté les méthodes ronronner() et grimper().

```java
package Ex4;

public class Chat extends Animal {
    public void crier() {
        System.out.println("Le chat fait un cri.");
    }
    public void ronronner() {
        System.out.println("Le chat ronronne.");
    }
    public void grimper() {
        System.out.println("Le chat grimpe.");
    }
}
```

Classe Main

Dans le Main, j'ai illustré le polymorphisme en déclarant deux variables de type Animal : a1 référençant un objet Chien et a2 référençant un objet Chat. Lors de l'appel de crier(), Java a exécuté la version redéfinie dans chaque classe fille, ce qui démontre le polymorphisme dynamique.

J'ai aussi créé des objets monChien et monChat de leurs types concrets pour accéder aux méthodes spécifiques comme garder(), jouer(), ronronner() et grimper().

```java
package Ex4;

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Chien();
        Animal a2 = new Chat();

        a1.crier();
        a2.crier();

        a1.manger();
        a2.dormir();

        Chien monChien = new Chien();

        monChien.garder();
        monChien.jouer();

        Chat monChat = new Chat();

        monChat.ronronner();
        monChat.grimper();
    }
}
```

Résultat :

```
Le chien fait un cri.
Le chat fait un cri.
L'animal est en train de manger.
L'animal est en train de dormir.
Le chien garde.
Le chien joue
Le chat ronronne.
Le chat grimpe.
```

Exercice 5 : Polymorphisme – Figures géométriques

Dans cet exercice, j'ai modélisé des figures géométriques manipulées de manière uniforme en appliquant le polymorphisme.

Classe Figure

J'ai créé une classe Figure avec trois méthodes : dessiner() qui affiche un message générique, deplacer(x, y) qui affiche la nouvelle position de la figure, et redimensionner(facteur) qui affiche le facteur de redimensionnement.

```java
package Ex5;

public class Figure {
    public void dessiner() {
            System.out.println("Dessin d'une figure.");
        }
    public void deplacer(int x, int y) {
            System.out.println("Figure déplacée vers x=" + x + " y=" + y);
        }
    public void redimensionner(double facteur) {
            System.out.println("Figure redimensionnée avec facteur=" + facteur);
        }
}
```

Classe Cercle

J'ai créé une classe Cercle qui hérite de Figure avec un attribut rayon. J'ai redéfini la méthode dessiner() pour afficher un message spécifique au cercle. J'ai aussi ajouté calculerPerimetre() qui retourne 2 * π * rayon, et calculerSurface() qui retourne π * rayon².

```
package Ex5;

public class Cercle extends Figure {
    double rayon;

    Cercle(double rayon) {
        this.rayon = rayon;
    }

    public void dessiner() {
        System.out.println("Dessin d'un cercle de rayon=" + rayon
    }

    public double calculerPerimetre() {
        return 2 * Math.PI * rayon;
    }

    public double calculerSurface() {
        return Math.PI * rayon * rayon;
    }
}
```

Classe Triangle

J'ai créé une classe Triangle qui hérite de Figure avec les attributs base, hauteur, cote1 et cote2. J'ai redéfini dessiner() pour afficher un message spécifique au triangle. J'ai ajouté calculerPerimetre() qui retourne la somme des trois côtés, et calculerSurface() qui retourne (base * hauteur) / 2.

```
package Ex5;

public class Triangle extends Figure{
    double base;
    double hauteur;
    double cote1;
    double cote2;

    Triangle(double base, double hauteur, double cote1, double cote2) {
        this.base=base;
        this.hauteur=hauteur;
        this.cote1=cote1;
        this.cote2=cote2;
    }

    public void dessiner() {
        System.out.println("Dessin d'un triangle de base=" + base);
    }

    public double calculerPerimetre() {
        return cote1 + cote2 + base;
    }

    public double calculerSurface() {
        return (base * hauteur) / 2;
    }
}
```

Classe Main

Dans le Main, j'ai déclaré deux variables de type Figure : f1 référençant un Cercle et f2 référençant un Triangle. J'ai appelé dessiner(), deplacer() et redimensionner() sur les deux objets, ce qui illustre le polymorphisme dynamique. J'ai ensuite créé des objets concrets c et t pour appeler les méthodes calculerPerimetre() et calculerSurface() spécifiques à chaque figure.

```java
package Ex5;

public class MAIN {
        public static void main(String[] args) {

            Figure f1 = new Cercle(5);
            Figure f2 = new Triangle(4,3,3,4);

            System.out.println("Dessiner :");
            f1.dessiner();
            f2.dessiner();

            System.out.println("Déplacer :");

            f1.deplacer(10,20);
            f2.deplacer(5,15);
            System.out.println(" Redimensionner :");
            f1.redimensionner(2.0);
            f2.redimensionner(0.5);

            System.out.println(" Périmètre et Surface :");

            Cercle c = new Cercle(5);

            System.out.println("Périmètre cercle ="+c.calculerPerimetre());
            System.out.println("Surface cercle = "+c.calculerSurface());

            Triangle t = new Triangle(4, 3, 3, 4);
            System.out.println("Périmètre triangle = "+t.calculerPerimetre());
            System.out.println("Surface triangle   = "+t.calculerSurface());
        }
    }
```

Résultat :

```
Dessiner :
Dessin d'un cercle de rayon=5.0
Dessin d'un triangle de base=4.0
Déplacer :
Figure déplacée vers x=10 y=20
Figure déplacée vers x=5 y=15
 Redimensionner :
Figure redimensionnée avec facteur=2.0
Figure redimensionnée avec facteur=0.5
 Périmètre et Surface :
Périmètre cercle =31.41592653589793
Surface cercle = 78.53981633974483
Périmètre triangle = 11.0
Surface triangle   = 6.0
```

Exercice 7 : Classe abstraite et calculs géométriques

Dans cet exercice, j'ai structuré des formes géométriques en utilisant une classe abstraite pour imposer une architecture commune.

Classe abstraite Forme

J'ai créé une classe abstraite Forme avec une méthode abstraite calculerSurface() que chaque classe fille doit obligatoirement implémenter. J'ai aussi ajouté deux méthodes concrètes : afficherDescription() qui affiche une description générique et appelle calculerSurface(), et comparerSurface(Forme f) qui compare la surface de deux formes et affiche laquelle est la plus grande.

```java
package Ex7;

public abstract class Forme {

    public abstract double calculerSurface();

    public void afficherDescription() {
        System.out.println("Je suis une forme géométrique.");
        System.out.println("la surface est = "+calculerSurface());
    }

    public void comparerSurface(Forme f) {
        if (this.calculerSurface() > f.calculerSurface())
            System.out.println("Cette forme est plus grande.");
         else if (this.calculerSurface() < f.calculerSurface())
            System.out.println("Cette forme est plus petite.");
        else
            System.out.println("Les deux formes ont la même surface.");
    }
}
```

Classe Rectangle

J'ai créé une classe Rectangle qui hérite de Forme avec les attributs longueur et largeur. J'ai implémenté la méthode abstraite calculerSurface() qui retourne longueur * largeur. J'ai aussi ajouté calculerPerimetre() qui retourne 2 * (longueur + largeur), et validerDimensions() qui vérifie que les deux dimensions sont positives.

```java
package Ex7;

public class Rectangle extends Forme{
    double longueur;
    double largeur;

    Rectangle(double longueur, double largeur) {
        this.longueur = longueur;
        this.largeur  = largeur;
    }

    public double calculerSurface() {
        return longueur*largeur;
    }

    public double calculerPerimetre() {
        return 2*(longueur + largeur);
    }

    public boolean validerDimensions() {
        if (longueur > 0 && largeur > 0) {
            System.out.println("Dimensions du rectangle valides.");
            return true;
        }else {
            System.out.println("Dimensions invalides !");
            return false;
        }
    }
```

Classe Cercle

J'ai créé une classe Cercle qui hérite de Forme avec l'attribut rayon. J'ai implémenté calculerSurface() qui retourne π * rayon². J'ai aussi ajouté calculerPerimetre() qui retourne 2 * π * rayon, et validerDimensions() qui vérifie que le rayon est positif.

```java
package Ex7;

public class Cercle extends Forme {
    double rayon;

    Cercle(double rayon) {
        this.rayon = rayon;
    }

    public double calculerSurface() {
        return Math.PI * rayon * rayon;
    }

    public double calculerPerimetre() {
        return 2 * Math.PI * rayon;
    }

    public boolean validerDimensions() {
        if (rayon > 0) {
            System.out.println("Rayon du cercle valide.");
            return true;
        } else {
            System.out.println("Rayon invalide !");
            return false;
        }
    }
}
```

Classe Main

Dans le Main, j'ai créé un objet Rectangle de dimensions 5x3 et un objet Cercle de rayon 4. J'ai testé validerDimensions() pour les deux objets, appelé afficherDescription() qui utilise le polymorphisme pour appeler la bonne version de calculerSurface(), affiché les périmètres, et comparé les surfaces des deux formes via comparerSurface().

```java
package Ex7;

public class Cercle extends Forme {
    double rayon;

    Cercle(double rayon) {
        this.rayon = rayon;
    }

    public double calculerSurface() {
        return Math.PI * rayon * rayon;
    }

    public double calculerPerimetre() {
        return 2 * Math.PI * rayon;
    }

    public boolean validerDimensions() {
        if (rayon > 0) {
            System.out.println("Rayon du cercle valide.");
            return true;
        } else {
            System.out.println("Rayon invalide !");
            return false;
        }
    }
}
```

Résultat :

```
Validation
Dimensions du rectangle valides.
Rayon du cercle valide.
 Description
Je suis une forme géométrique.
la surface est = 15.0
Je suis une forme géométrique.
la surface est = 50.26548245743669
Périmètre
Périmètre rectangle = 16.0
Périmètre cercle    = 25.132741228718345
 Comparaison des surfaces
Cette forme est plus petite.
```

Exercice 8 : Interface – Entités capables de voler

Dans cet exercice, j'ai modélisé des entités volantes en utilisant une interface pour définir un contrat commun.

Interface Volant

J'ai créé une interface Volant avec trois méthodes abstraites : voler(), atterrir() et changerAltitude(int altitude).

```
package Ex8;

public interface Volant {

    public void voler();
    public void atterrir();
    public void changerAltitude(int altitude);
}
```

Classe Oiseau

J'ai créé une classe Oiseau qui implémente l'interface Volant. J'ai implémenté les trois méthodes de l'interface avec des comportements spécifiques à l'oiseau.

```
package Ex8;

public class Oiseau implements Volant {

    private String nom;

    public Oiseau(String nom) {
        this.nom=nom;
    }
    public void voler() {
        System.out.println(nom+" s'envole !");
    }
    public void atterrir() {
        System.out.println(nom+" se pose.");
    }
    public void changerAltitude(int altitude) {
        System.out.println(nom+" vole à "+altitude+" m.");
    }
    public void migrer() {
        System.out.println(nom+" migre");
    }
    public void construireNid() {
        System.out.println(nom+" construit un nid");
    }
}
```

Classe Avion

J'ai créé une classe Avion qui implémente l'interface Volant avec les attributs nom et altitude. J'ai implémenté les trois méthodes de l'interface, et j'ai ajouté embarquerPassager() et afficherAltitude() pour des comportements spécifiques à l'avion.

```java
package Exo;

public class Avion implements Volant  {

    private String nom;
    private int altitude;

    public Avion(String nom) {
        this.nom = nom;
        this.altitude = 0;
    }

    public void voler() {
        System.out.println(nom+" décolle");
    }

    public void atterrir() {
        System.out.println(nom+" atterrit.");
        this.altitude=0;
    }

    public void changerAltitude(int altitude) {
        this.altitude = altitude;
        System.out.println(nom+" vole à "+altitude+" m.");
    }

    public void embarquerPassager(){
        System.out.println("Les passagers embarquent dans "+nom+".");
    }

    public void afficherAltitude() {
        System.out.println(nom+" est à une altitude de "+altitude+" m.");
    }
```

Classe Main

Dans le Main, j'ai d'abord testé les objets Oiseau et Avion individuellement pour vérifier leurs méthodes spécifiques. Ensuite, j'ai illustré le polymorphisme en créant un tableau de type Volant[] contenant un Oiseau et un Avion. J'ai parcouru ce tableau avec une boucle for et appelé voler(), changerAltitude() et atterrir() sur chaque élément, ce qui démontre que l'interface permet de manipuler des objets différents de manière uniforme.

```java
public class Main {
        public static void main(String[] args) {

            System.out.println("** TEST OISEAU");

            Oiseau oiseau = new Oiseau("zizo");

            oiseau.construireNid();
            oiseau.voler();
            oiseau.changerAltitude(250);
            oiseau.migrer();
            oiseau.atterrir();

            System.out.println("** TEST AVION");

            Avion avion = new Avion("Rayanair");

            avion.embarquerPassager();
            avion.voler();
            avion.changerAltitude(11000);
            avion.afficherAltitude();
            avion.atterrir();
            avion.afficherAltitude();

            System.out.println("**TEST POLYMORPHISME ");

            Volant[] volants = new Volant[2];
            volants[0] = new Oiseau("cocoo");
            volants[1] = new Avion("Air Arabia");

            for (Volant v : volants) {
                v.voler();
                v.changerAltitude(1000);
                v.atterrir();
```

Résultat :

```
** TEST OISEAU
zizo construit un nid
zizo s'envole !
zizo vole à 250 m.
zizo migre
zizo se pose.
** TEST AVION
Les passagers embarquent dans Rayanair.
Rayanair décolle
Rayanair vole à 11000 m.
Rayanair est à une altitude de 11000 m.
Rayanair atterrit.
Rayanair est à une altitude de 0 m.
**TEST POLYMORPHISME
cocoo s'envole !
cocoo vole à 1000 m.
cocoo se pose.
Air Arabia décolle
Air Arabia vole à 1000 m.
Air Arabia atterrit.
```

Exercice 9 : Héritage, interface et abstraction – Véhicules modernes

Dans cet exercice, j'ai combiné les trois concepts : classe abstraite, interface et héritage pour concevoir un système de véhicules modernes.

Classe abstraite Vehicule

J'ai créé une classe abstraite Vehicule avec un attribut protégé vitesse. J'ai implémenté deux méthodes concrètes : accelerer(int v) qui augmente la vitesse et freiner(int v) qui la diminue.

```java
package Ex9;

public abstract class Vehicule {

    protected int vitesse;

    public Vehicule(int vitesse) {
            this.vitesse=vitesse;
    }

    public void accelerer(int v) {
            this.vitesse += v;
            System.out.println("Accélération : Vitesse : " + vitesse + " km/h");
    }

    public void freiner(int v) {
            this.vitesse -= v;
            System.out.println("Freinage : Vitesse : " + vitesse + " km/h");
    }

    }
```

Interface Electrique

J'ai créé une interface Electrique avec deux méthodes abstraites : charger() et verifierBatterie(). Cette interface représente le contrat que toute entité électrique doit respecter.

```java
package Ex9;

public interface Electrique {
    public void charger();
    public void verifierBatterie();
}
```

Classe VoitureElectrique

J'ai créé une classe VoitureElectrique qui hérite de Vehicule et implémente l'interface Electrique. Elle possède deux attributs supplémentaires : autonomie et batterie. J'ai implémenté les méthodes de l'interface charger() qui remet la batterie à 100%, et verifierBatterie() qui affiche le niveau de batterie. J'ai aussi

ajouté afficherAutonomie() et optimiserConsommation() comme comportements spécifiques.

```java
package Ex9;

public class VoitureElectrique extends Vehicule implements Electrique {

    private int autonomie;
    private int batterie;

    public VoitureElectrique(int vitesse, int autonomie, int batterie) {
        super(vitesse);
        this.autonomie = autonomie;
        this.batterie  = batterie;
    }

    public void charger() {
        this.batterie = 100;
        System.out.println("Batterie chargée.");
    }

    public void verifierBatterie() {
        System.out.println("Niveau batterie : "+batterie + "%");
    }

    public void afficherAutonomie() {
        System.out.println("Autonomie restante : "+autonomie+" km.");
    }

    public void optimiserConsommation() {
        System.out.println("Consommation optimisée.");
    }

}
```

Classe Moto

J'ai créé une classe Moto qui hérite de Vehicule avec un attribut type. J'ai ajouté deux méthodes spécifiques : faireRoueArriere() et afficherTypeMoto().

```java
package Ex9;

public class Moto extends Vehicule {

    private String type;

    public Moto(int vitesse, String type) {
        super(vitesse);
        this.type = type;
    }

    public void faireRoueArriere() {
        System.out.println("Roue arrière ! Vitesse : " + vitesse + " km/h");
    }

    public void afficherTypeMoto() {
        System.out.println("Type de moto : " + type);
    }

}
```

Classe Main

Dans le Main, j'ai créé un objet VoitureElectrique avec une vitesse initiale de 0, une autonomie de 350 km et une batterie à 80%. J'ai testé toutes ses méthodes : accélération, vérification de batterie, autonomie, optimisation et chargement. J'ai

aussi créé un objet Moto de type "Sportive" et testé ses méthodes spécifiques. Cet exercice m'a permis d'illustrer comment une classe peut à la fois hériter d'une classe abstraite et implémenter une interface.

```
package Ex9;

public class Main {
    public static void main(String[] args) {

        System.out.println("** Voiture Electrique");
        VoitureElectrique ve = new VoitureElectrique(0,350,80);
        ve.accelerer(50);
        ve.verifierBatterie();
        ve.afficherAutonomie();
        ve.optimiserConsommation();
        ve.charger();
        ve.verifierBatterie();
        ve.freiner(20);

        System.out.println("** MOTO ");
        Moto moto = new Moto(0,"Sportive");
        moto.accelerer(80);
        moto.faireRoueArriere();
        moto.afficherTypeMoto();
        moto.freiner(30);
    }
}
```

Résultat :

```
** Voiture Electrique
Accélération : Vitesse : 50 km/h
Niveau batterie : 80%
Autonomie restante : 350 km.
Consommation optimisée.
Batterie chargée.
Niveau batterie : 100%
Freinage : Vitesse : 30 km/h
** MOTO
Accélération : Vitesse : 80 km/h
Roue arrière ! Vitesse : 80 km/h
Type de moto : Sportive
Freinage : Vitesse : 50 km/h
```

Exercice 10 : Cas d'étude – Système de gestion d'une bibliothèque numérique

Dans cet exercice, j'ai conçu un système complet de gestion de bibliothèque en combinant les classes abstraites, les interfaces, l'héritage et le polymorphisme.

Classe abstraite Document

J'ai créé une classe abstraite Document avec quatre attributs protégés : identifiant, titre, auteur et disponible. J'ai implémenté les méthodes concrètes emprunter() qui vérifie la disponibilité avant d'emprunter, retourner() qui remet le document à disponible, et afficherInformations() qui affiche les informations générales. J'ai déclaré la méthode abstraite calculerDureeEmpruntMax() que chaque classe fille doit obligatoirement implémenter.

```java
package Ex10;

public abstract class Document {

    protected String identifiant;
    protected String titre;
    protected String auteur;
    protected boolean disponible;

    public Document(String identifiant, String titre, String auteur) {
        this.identifiant = identifiant;
        this.titre       = titre;
        this.auteur      = auteur;
        this.disponible  = true;
    }
    public void emprunter() {
        if (disponible) {
            disponible = false;
            System.out.println("Document '" + titre + "' emprunté avec succès.");
        } else
            System.out.println("Document '" + titre + "' non disponible.");
    }
    public void retourner() {
        disponible = true;
        System.out.println("Document '" + titre + "' retourné avec succès.");
    }
    public void afficherInformations() {
        System.out.println("ID : " + identifiant);
        System.out.println("Titre : " + titre);
        System.out.println("Auteur : " + auteur);
        System.out.println("Disponible : " + (disponible ? "Oui" : "Non"));
    }
    public abstract int calculerDureeEmpruntMax();
}
```

Interfaces Empruntable et Consultable

J'ai créé une interface Empruntable avec deux méthodes : estDisponible() et reserver(). J'ai aussi créé une interface Consultable avec consulter() et afficherResume(). Ces interfaces définissent les contrats selon le type de document.

```java
package Ex10;

public interface Empruntable {
    boolean estDisponible();
    void reserver();
}
```

Classe Livre

J'ai créé une classe Livre qui hérite de Document et implémente Empruntable. J'ai ajouté les attributs nombreDePages et genre. J'ai implémenté calculerDureeEmpruntMax() qui retourne 21 jours, estDisponible() et reserver(). J'ai aussi ajouté afficherDetails() et recommander().

```java
package Ex10;

public class Livre extends Document implements Empruntable {

        private int nombreDePages;
        private String genre;

        public Livre(String identifiant, String titre, String auteur,int nombreDePages, String genre) {
            super(identifiant,titre,auteur);
            this.nombreDePages=nombreDePages;
            this.genre=genre;
        }
        public int calculerDureeEmpruntMax() {
            return 21;
        }
        public boolean estDisponible() {
            return disponible;
        }
        public void reserver() {
            if (disponible) {
                System.out.println("Livre \"" +titre+"\" réservé.");
            } else {
                System.out.println("Livre \"" +titre+ "\"déjà réservé.");
            }
        }
        public void afficherDetails() {
            afficherInformations();
            System.out.println("Pages : "+nombreDePages);
            System.out.println("Genre : "+genre);
            System.out.println("Durée max emprunt : " +calculerDureeEmpruntMax() + " jours");
        }
        public void recommander() {
            System.out.println("Livre recommandé : \"" +titre+ "\" de " +auteur);
        }
    }
```

Classe Magazine

J'ai créé une classe Magazine qui hérite de Document et implémente Empruntable. J'ai ajouté les attributs numero et datePublication. J'ai implémenté calculerDureeEmpruntMax() qui retourne 7 jours, et j'ai ajouté estRecent() qui vérifie si le magazine a été publié dans le dernier mois, et afficherEdition() pour afficher les détails de l'édition.

```java
package Ex10;

import java.time.LocalDate;
public class Magazine extends Document implements Empruntable {

     private int numero;
     private LocalDate datePublication;

     public Magazine(String identifiant, String titre, String auteur,int numero,LocalDate datePublication) {
     super(identifiant,titre,auteur);
     this.numero=numero;
     this.datePublication=datePublication;
     }
     public int calculerDureeEmpruntMax() {
        return 7;
     }
     public boolean estDisponible() {
        return disponible;
     }
     public void reserver() {
        if (disponible)
            System.out.println("Magazine \"" +titre+"\"réservé.");
       else
            System.out.println("Magazine \"" +titre+"\" déjà réservé.");
     }
        public boolean estRecent() {
            return datePublication.isAfter(LocalDate.now().minusMonths(1));
        }
        public void afficherEdition() {
            afficherInformations();
            System.out.println("Numéro : " + numero);
            System.out.println("Date publication : " + datePublication);
            System.out.println("Récent : " + (estRecent() ? "Oui" : "Non"));
            System.out.println("Durée max : " + calculerDureeEmpruntMax() + " jours");
       }
}
```

Classe DocumentNumerique

J'ai créé une classe DocumentNumerique qui hérite de Document et implémente Consultable. J'ai ajouté les attributs format et tailleFichier. J'ai implémenté calculerDureeEmpruntMax() qui retourne 14 jours, consulter() et afficherResume(). J'ai aussi ajouté telecharger() et afficherTaille().

```java
package Ex10;


public class DocumentNumerique extends Document implements Consultable{

    private String format;
    private double tailleFichier;

    public DocumentNumerique(String identifiant,String titre,String auteur,String format,double tailleFichier) {
        super(identifiant, titre, auteur);
        this.format=format;
        this.tailleFichier=tailleFichier;
    }

    public int calculerDureeEmpruntMax() {
        return 14;
    }

    public void consulter() {
        System.out.println("Consultation de \"" + titre + "\" en ligne.");
    }

    public void afficherResume() {
        System.out.println("Résumé : \"" + titre + "\" par " + auteur);
    }

    public void telecharger() {
        System.out.println("Téléchargement de \"" + titre + "\"(" +format+")");
    }
    public void afficherTaille() {
        System.out.println("Taille : " + tailleFichier + " Mo");
    }

}
```

Classe Utilisateur

J'ai créé une classe Utilisateur avec les attributs identifiant, nom et une liste historique de type ArrayList<Document>. Dans la méthode emprunterDocument(), j'ai utilisé instanceof pour vérifier si le document implémente Empruntable avant de l'emprunter et de l'ajouter à l'historique. J'ai aussi implémenté retournerDocument() et afficherHistorique().

```java
package Ex10;
import java.util.ArrayList;

public class Utilisateur {
    private String identifiant;
    private String nom;
    private ArrayList<Document> historique;

    public Utilisateur(String identifiant, String nom) {
        this.identifiant=identifiant;
        this.nom=nom;
        this.historique=new ArrayList<>();
    }
    public void emprunterDocument(Document d) {
        if (d instanceof Empruntable) {
            Empruntable e = (Empruntable) d;
            if (e.estDisponible()) {
                d.emprunter();
                historique.add(d);
            } else
                System.out.println("Document '" + d.titre + "' non disponible.");
        } else
            System.out.println("Ce document n'est pas empruntable.");
    }
    public void retournerDocument(Document d) {
        d.retourner();
    }
    public void afficherHistorique() {
        System.out.println(" Historique de "+nom );
        if (historique.isEmpty()) {
            System.out.println("Aucun document emprunté.");
        } else {
            for (Document d : historique) {
                System.out.println("- "+ d.titre + " (" + d.auteur + ")");
            }
        }
```

Classe Main

Dans le Main, j'ai créé trois documents : un Livre, un Magazine et un DocumentNumerique. J'ai utilisé un tableau de type Document[] pour les manipuler de manière uniforme via le polymorphisme, en affichant leurs informations et leurs durées d'emprunt maximales. J'ai ensuite créé un utilisateur Ali et testé les opérations d'emprunt, de retour et d'affichage de l'historique. Enfin, j'ai appelé les méthodes spécifiques à chaque type de document comme recommander(), afficherEdition(), telecharger() et afficherTaille().

```java
package Ex10;

import java.time.LocalDate;

public class Main {
    public static void main(String[] args) {

        Livre livre = new Livre("L001","Le Petit Prince","Saint-Exupéry",96,"Roman");

        Magazine magazine = new Magazine("M001", "Science & Vie","Dupont",42,LocalDate.of(2026,4,1));

        DocumentNumerique doc = new DocumentNumerique("D001", "Java POO","Martin", "PDF", 5.2);

        Document[] documents = { livre, magazine, doc };
        System.out.println("INFORMATIONS DES DOCUMENTS");
        for (Document d : documents) {
            d.afficherInformations();
            System.out.println("Durée max emprunt : "+d.calculerDureeEmpruntMax()+" jours");
            System.out.println("---");
        }
        System.out.println("EMPRUNT");
        Utilisateur user = new Utilisateur("U001", "Ali");
        user.emprunterDocument(livre);
        user.emprunterDocument(magazine);
        user.emprunterDocument(doc);
        System.out.println("RETOUR");
        user.retournerDocument(livre);
        System.out.println(" HISTORIQUE");
        user.afficherHistorique();
        System.out.println("COMPORTEMENTS SPECIFIQUES");
        livre.recommander();
        magazine.afficherEdition();
        doc.telecharger();
        doc.afficherTaille();
    }
}
```

Resultat :

```
INFORMATIONS DES DOCUMENTS
ID : L001
Titre : Le Petit Prince
Auteur : Saint-Exupéry
Disponible : Oui
Durée max emprunt : 21 jours
---
ID : M001
Titre : Science & Vie
Auteur : Dupont
Disponible : Oui
Durée max emprunt : 7 jours
---
ID : D001
Titre : Java POO
Auteur : Martin
Disponible : Oui
Durée max emprunt : 14 jours
---
EMPRUNT
Document 'Le Petit Prince' emprunté avec succès.
Document 'Science & Vie' emprunté avec succès.
Ce document n'est pas empruntable.
RETOUR
Document 'Le Petit Prince' retourné avec succès.
 HISTORIQUE
 Historique de Ali
- Le Petit Prince (Saint-Exupéry)
- Science & Vie (Dupont)
COMPORTEMENTS SPECIFIQUES
Livre recommandé : "Le Petit Prince" de Saint-Exupéry
ID : M001
Titre : Science & Vie
Auteur : Dupont
```